

Lab 4: Classical Machine Learning

QML

1 introduction

before we can do quantum ML, we need to make sure u have a solid handle on classical ML. this lab is a refresher (or introduction, depending on ur background) on the key classical techniques that have quantum analogues.

specifically, we're focusing on:

- support vector machines (SVMs) — bcs quantum kernel methods r a direct generalization
- kernel methods — bcs the kernel trick is exactly wt quantum feature maps exploit
- cross-validation — bcs u need this to evaluate any model properly
- decision boundaries — bcs visualizing them helps build intuition

we'll use scikit-learn (sklearn) for everything. if u don't have it: `pip install scikit-learn matplotlib numpy`.

Key Point. the whole point of this lab is to set up the classical baselines we'll compare against in lab 5. so take it seriously — ur quantum classifier needs to beat (or at least match) the classical SVM to be interesting.

2 the dataset

we're using the iris dataset. it's a classic ML dataset with:

- 150 samples
- 4 features: sepal length, sepal width, petal length, petal width
- 3 classes: setosa, versicolor, virginica

for most of our experiments we'll simplify things:

- use only 2 features (petal length and petal width) for visualization
- use only 2 classes (versicolor and virginica) for binary classification

this gives us a nice 2D binary classification problem that's easy to visualize and still nontrivial (the two classes overlap a bit).

Exercise 1. loading and exploring the data.

1. load the iris dataset using `sklearn.datasets.load_iris()`

2. extract only classes 1 and 2 (versicolor and virginica)
3. extract only features 2 and 3 (petal length and petal width) — these r the most discriminative
4. standardize the features using `sklearn.preprocessing.StandardScaler` (subtract mean, divide by std)
5. plot the 2D scatter plot with different colors for each class
6. split into 70% train and 30% test using `train_test_split` with `random_state=42`
7. report: how many training samples? how many test samples? r the classes balanced?

3 support vector machines

3.1 the idea

an SVM finds the hyperplane that separates the two classes with maximum margin. the “support vectors” r the data points closest to the decision boundary — they r the ones that actually determine the boundary.

Definition 3.1. given training data $\{(x_i, y_i)\}$ where $y_i \in \{-1, +1\}$, the linear SVM solves:

$$\min_{w,b} \frac{1}{2} \|w\|^2 \quad \text{subject to} \quad y_i(w \cdot x_i + b) \geq 1 \quad \forall i$$

the decision boundary is $w \cdot x + b = 0$ and the margin is $\frac{2}{\|w\|}$.

in practice we use the soft-margin version which allows some misclassification. the regularization parameter C controls the tradeoff: high C = less tolerance for misclassification, low C = wider margin but more errors.

Exercise 1. linear SVM.

1. import `SVC` from `sklearn.svm`
2. train a linear SVM on the training data: `SVC(kernel='linear', C=1.0)`
3. call `.fit(X_train, y_train)`
4. compute accuracy on both train and test sets using `.score()`
5. how many support vectors does the model have? (check `.support_vectors_`)
6. visualize the decision boundary. here’s how:
 - create a mesh grid over the feature space using `np.meshgrid`
 - predict the class for each point in the grid
 - use `plt.contourf` to color the regions
 - overlay the training data points
 - highlight the support vectors
7. try $C = 0.01, 0.1, 1, 10, 100$. how does the decision boundary change?

4 the kernel trick

4.1 why we need kernels

wt if the data isn't linearly separable? one option: map the data to a higher-dimensional space where it is linearly separable.

Example 4.1. consider 1D data: class -1 at $x \in [-2, -1] \cup [1, 2]$ and class $+1$ at $x \in [-0.5, 0.5]$. this isn't linearly separable in 1D. but map $x \rightarrow (x, x^2)$ and suddenly it is! the classes r separated by a horizontal line in the 2D space.

the problem: computing in high-dimensional spaces is expensive. the kernel trick avoids this by computing inner products in the high-dimensional space without explicitly mapping to it.

Definition 4.1. a kernel function $k(x_i, x_j) = \phi(x_i) \cdot \phi(x_j)$ computes the inner product of the mapped features without computing ϕ explicitly. common kernels:

- linear: $k(x, x') = x \cdot x'$
- polynomial: $k(x, x') = (x \cdot x' + c)^d$
- RBF (Gaussian): $k(x, x') = \exp(-\gamma \|x - x'\|^2)$

Intuition. the RBF kernel implicitly maps to an infinite-dimensional space. that's wild. and bcs of the kernel trick, computing with it is no harder than computing with a finite kernel. this is wt makes kernel methods so powerful — and it's the same principle behind quantum kernels. a quantum computer can compute kernels in exponentially large feature spaces.

Exercise 1. kernel comparison.

1. train SVMs with different kernels on the iris data:

- linear: `SVC(kernel='linear')`
- polynomial degree 2: `SVC(kernel='poly', degree=2)`
- polynomial degree 3: `SVC(kernel='poly', degree=3)`
- RBF: `SVC(kernel='rbf')`

2. for each kernel, report train and test accuracy

3. visualize the decision boundary for each kernel (4 subplots)

4. which kernel gives the best test accuracy?

5. which kernel has the most complex decision boundary?

6. is there evidence of overfitting for any kernel?

Exercise 2. kernel matrices. the kernel matrix (or Gram matrix) K has entries $K_{ij} = k(x_i, x_j)$. it captures the similarity structure of ur data as seen through the kernel.

1. compute the kernel matrix for the RBF kernel: u can use `sklearn.metrics.pairwise.rbf_kernel(X_train, gamma=1.0)`

2. visualize it as a heatmap using `plt.imshow`
3. try different values of γ (0.1, 1.0, 10.0). how does the kernel matrix change?
4. what does a “good” kernel matrix look like for classification? (hint: u want high similarity within classes and low similarity between classes)
5. this is super relevant for quantum ML bcs the quantum kernel is literally just another kernel matrix. the whole question is whether the quantum kernel captures useful similarity that classical kernels miss.

4.2 precomputed kernels

in quantum ML, we’ll compute the kernel matrix on the quantum computer and then feed it to a classical SVM. sklearn supports this with the `precomputed` kernel option.

Exercise 3. SVM with precomputed kernel. this exercise practices the exact workflow we’ll use in lab 5 with quantum kernels.

1. compute the RBF kernel matrix for the training data: $K_{\text{train}} = k(X_{\text{train}}, X_{\text{train}})$
2. compute the test kernel matrix: $K_{\text{test}} = k(X_{\text{test}}, X_{\text{train}})$ (note: test vs train, not test vs test!)
3. train an SVM with precomputed kernel: `SVC(kernel='precomputed')`
4. call `.fit(K_train, y_train)`
5. predict using `.predict(K_test)`
6. verify that the accuracy is the same as using `SVC(kernel='rbf')` directly

what u just did: computed the kernel matrix separately and fed it to the SVM. in lab 5, we’ll replace the RBF kernel computation with a quantum kernel computation. the SVM part stays exactly the same.

5 cross-validation

never evaluate ur model on the training data only. that’s a recipe for fooling urself.

Definition 5.1. k -fold cross-validation splits the training data into k equal folds. for each fold i : train on all folds except i , test on fold i . report the average performance across all k folds.

Exercise 1. cross-validation.

1. use `sklearn.model_selection.cross_val_score` to evaluate the RBF SVM with 5-fold cross-validation
2. report the mean and standard deviation of the accuracy across folds
3. now do cross-validation to select the best hyperparameters. use `GridSearchCV` with:
 - C: [0.01, 0.1, 1, 10, 100]

- `gamma`: [0.01, 0.1, 1, 10]
4. with the best C and γ values?
 5. train a final model with these best parameters and evaluate on the held-out test set
 6. is the test accuracy similar to the cross-validation accuracy? if not, you might have a problem.

6 beyond SVM: other classifiers for comparison

Exercise 1. classifier comparison. train these classifiers on the iris data and compare:

1. logistic regression: `LogisticRegression()` from `sklearn.linear_model`
2. k -nearest neighbors: `KNeighborsClassifier(n_neighbors=5)` from `sklearn.neighbors`
3. decision tree: `DecisionTreeClassifier(max_depth=3)` from `sklearn.tree`
4. random forest: `RandomForestClassifier(n_estimators=100)` from `sklearn.ensemble`
5. RBF SVM (your best from above)

for each:

- run 5-fold cross-validation
- report mean accuracy $\pm$ std
- visualize the decision boundary

create a bar plot comparing all classifiers. which one wins?

7 feature engineering and dimensionality

this section connects classical feature engineering to quantum feature maps.

Exercise 1. feature maps.

1. take the 2D iris data (petal length, petal width)
2. create polynomial features up to degree 2 using `sklearn.preprocessing.PolynomialFeatures(degree=2)`.
this maps $(x_1, x_2) \rightarrow (1, x_1, x_2, x_1^2, x_1x_2, x_2^2)$.
3. train a linear SVM on the expanded features
4. compare its accuracy with the polynomial kernel SVM (degree 2) on the original features.
they should be similar!
5. this is the kernel trick in action: the polynomial kernel SVM is implicitly doing what you just did explicitly.
6. now try: what if you used all 4 iris features instead of just 2? does accuracy improve?

7. with all 4 features, how does the linear SVM compare to the RBF SVM?

the takeaway: feature maps and kernels r two sides of the same coin. quantum feature maps will encode classical data into quantum states, creating a kernel in an exponentially large Hilbert space.

8 the full pipeline

Exercise 1. putting it all together. build a complete ML pipeline for the binary iris classification task:

1. load and preprocess data (standardize, split)
2. use `GridSearchCV` to find the best SVM hyperparameters
3. train the final model
4. evaluate on the test set
5. compute and display the confusion matrix using `sklearn.metrics.confusion_matrix`
6. compute precision, recall, and F1 score using `classification_report`
7. visualize the decision boundary of the final model

record the following numbers — u'll need them in lab 5:

- best classical SVM test accuracy: _____
- best kernel type: _____
- best C : _____
- best γ (if RBF): _____

9 connecting to quantum ML

Exercise 1. reflection. think about and write brief answers to these questions:

1. in the kernel trick, we compute $k(x, x') = \phi(x) \cdot \phi(x')$ without computing ϕ explicitly. in quantum ML, $\phi(x)$ is a quantum state $|\phi(x)\rangle$ and the kernel is $k(x, x') = |\langle \phi(x) | \phi(x') \rangle|^2$. why might a quantum feature map ϕ be more powerful than a classical one?
2. the RBF kernel maps to infinite dimensions. the quantum kernel maps to 2^n dimensions (for n qubits). is the quantum kernel always better? wt factors matter?
3. wt's the bottleneck in computing a quantum kernel matrix? (hint: think about how many circuit evaluations u need for an $m \times m$ kernel matrix)
4. based on ur classical results, wt accuracy should the quantum classifier aim for to be considered “useful”?

10 submission

submit a jupyter notebook with:

- all exercises completed with code and output
- decision boundary plots for all classifiers
- the comparison bar plot
- the recorded classical baseline numbers
- ur reflection answers

these results r ur classical baselines for lab 5. save ur trained models and preprocessing steps — u'll need them.